Լաբորատոր աշխատանք 8

Ջրի սպառման վճար

Աշխատանքի նպատակ- ուսումնասիրել C++ լեզվում դասերի և կոնստրուկտորների կիրառումը, իրականացնել տվյալների ստուգում, սպառման և ընդհանուր վճարի հաշվարկում, ինչպես նաև զանգվածի միջոցով գտնել ամենաբարձր վճարով բաժանորդին:

Խնդիր- *Ստեղծել WaterBill դասը: Պահպանվող տվյալներն են՝ բաժանորդի կողը, սպառումը մ³-ով, մեկ մ³-ի սակագինը և սպասարկման վճարը: 1. Կոնստրուկտորներ: WaterBill()՝ ընդհանուր կանոններով, երեք պարամետրով՝ կող, սպառում, սակագին (սպասարկման վճարը՝ 0), չորս պարամետրով՝ բոլոր տվյալները: 2. Ստուգումներ: սպառումը՝ ոչ բացասական, սակագինը՝ դրական, սպասարկման վճարը՝ ոչ բացասական: Քանակները և օրերը ամբողջ թվեր են, չափումները, գները ու տոկոսները՝ double: 3. Փոփոխող մեթոդ: Ավելացնել դրական չափով սպառում: 4. Հաշվարկային մեթոդներ: Հաշվել սպառման արժեքը և ընդհանուր վճարը: 5. Զանգվածի մշակում: Ցուցադրել ամենաբարձր ընդհանուր վճարով բաժանորդին և ընդհանուր վճարների գումարը: Որոնման ժամանակ օգտագործել համապատասխան հաշվարկային մեթոդի արդյունքը: C++ • 4 6. Ստուգիչ օրինակ: Մուտք՝ սպառում՝ 15, սակագին՝ 200, սպասարկում՝ 500: Ակնկալվող արդյունք՝ սպառման արժեքը՝ 3000, ընդհանուր վճարը՝ 3500: Օրինակը վերաբերում է օբյեկտի սկզբնական վիճակին՝ մինչև փոփոխող մեթոդի կանչը:*

```
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

class WaterBill {
private:
string code;
double consumption;
double tariff;
double serviceFee;

public:
WaterBill() {
code = "Unknown";
consumption = 0;
tariff = 1.0;
serviceFee = 0;
}

WaterBill(string c, double cons, double tar) {
code = (c.empty() ? "Unknown" : c);

if (cons >= 0) {
```

```
consumption = cons;
} else {
consumption = 0;
cout << "[Error] Consumption cannot be negative. Set to 0.\n";
}

if (tar > 0) {
tariff = tar;
} else {
tariff = 1.0;
cout << "[Error] Tariff must be positive. Set to 1.0.\n";
}

serviceFee = 0;
}

WaterBill(string c, double cons, double tar, double sf) {
code = (c.empty() ? "Unknown" : c);

if (cons >= 0) {
consumption = cons;
} else {
consumption = 0;
cout << "[Error] Consumption cannot be negative. Set to 0.\n";
}

if (tar > 0) {
tariff = tar;
} else {
tariff = 1.0;
cout << "[Error] Tariff must be positive. Set to 1.0.\n";
}

if (sf >= 0) {
serviceFee = sf;
} else {
serviceFee = 0;
cout << "[Error] Service fee cannot be negative. Set to 0.\n";
}
}

void addConsumption(double extraCons) {
if (extraCons > 0) {
consumption += extraCons;
} else {
cout << "[Error] Added consumption must be positive. Object unchanged.\n";
}
}

double calculateConsumptionCost() const {
return consumption * tariff;
}

double calculateTotalBill() const {
return calculateConsumptionCost() + serviceFee;
}
```

```
void show() const {
cout << "Subscriber Code: " << code
<< " | Consumption: " << fixed << setprecision(2)
<< consumption << " m3"
<< " | Tariff: " << tariff
<< " | Service Fee: " << serviceFee
<< " | Consumption Cost: " << calculateConsumptionCost()
<< " | Total Bill: " << calculateTotalBill() << endl;
}

string getCode() const {
return code;
}
};

int main() {
cout << "=== VERIFICATION EXAMPLE ===" << endl;

WaterBill testObj("Test01", 15.0, 200.0, 500.0);
testObj.show();

cout << "Expected: Consumption Cost = 3000, Total Bill = 3500\n\n";

cout << "=== CLASS DEMONSTRATION ===" << endl;

WaterBill obj1;
WaterBill obj2("A101", 25.5, 180.0);
WaterBill obj3("B202", 40.0, 190.0, 350.0);

cout << "\nInitial objects:" << endl;
obj1.show();
obj2.show();
obj3.show();

cout << "\nAdding 10 m3 to subscriber A101:" << endl;
obj2.addConsumption(10.0);
obj2.show();

cout << "\nTesting invalid consumption:" << endl;
obj2.addConsumption(-5.0);
obj2.show();

cout << "\nSubscriber Code: " << obj3.getCode() << endl;

return 0;
}
```